NAME: D.VAISHNAVI

PROGRAMMING IN C++
ASSIGNMENT 2 – CO2

CO2 Coverage: function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual **Total Marks:** 50 **Bloom's Level:** K5 – Evaluate

Compiler: g++ 13.3 (Ubuntu). **Compile command:** g++ -std=c++17 -Wall -Wextra file.cpp -o file (timing programs also use -O0 and -O2).

GitHub Repository Link: https://github.com/vaishu1104/DSA-0118-192524143-Assignment2

Q1 – Call by Value, by Reference and by Pointer

1. Prediction (made before running)

Experiment	Predicted result	Reason
Fee after call by value	45000	The function changes its own copy; the caller's fee is untouched.
Fee after call by reference	40500	The parameter is another name for the caller's fee: 45000 − 4500.
Fee after call by pointer	40500	*fee is the caller's variable, so it is reduced in place.
Predict-the-output, line 1	1 12 13	a is passed by value (copy, stays 1); b is a reference (12); c is reached through the pointer (13).
Predict-the-output, line 2	1 100	bigger(a, c) compares 1 and 13 and returns a reference to c; assigning 100 changes c.
Predict-the-output, line 3	16 13 6 (then newline)	show(1) = 1+5+10 (y, z defaulted); show(1,2) = 1+2+10 (z defaulted); show(1,2,3) = 6 (no defaults).

The given program was run unchanged; the actual output matched the prediction on every line (section 3).

2. Code

q1_three_ways.cpp

```
#include <iostream>
using namespace std;

void byValue(long long fee, int p) { fee -= fee * p / 100; cout << "  inside byValue: " << fee <<
'\n'; }
void byRef(long long& fee, int p)   { fee -= fee * p / 100; }
void byPtr(long long* fee, int p)   { *fee -= *fee * p / 100; }
long long byReturn(long long fee, int p) { return fee - fee * p / 100; }   // alternative

int main()
{
    long long a = 45000, b = 45000, c = 45000;
    byValue(a, 10);  cout << "by value   : " << a << '\n';
    byRef(b, 10);    cout << "by ref     : " << b << '\n';
```

```
    byPtr(&c, 10);   cout << "by pointer : " << c << '\n';

    // several concessions: merit 10%, sports 15%, single parent 25%
    long long fee = byReturn(byReturn(byReturn(45000, 10), 15), 25);
    cout << "all three concessions (return style): " << fee << '\n';
}
```

q1_complete.cpp

```
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;          // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;          // TODO 1 done: call by reference changes the caller's fee
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Complete the Code: TODO 1 was replaced by fee -= fee * percent / 100; – the same statement as in applyByValue, but the parameter is now a long long&.

3. Output

q1_predict.cpp — code

```cpp
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q1_predict.cpp -o q1_predict && ./q1_predict
1 12 13
1 100
16 13 6
```

q1_three_ways.cpp — code

```cpp
#include <iostream>
using namespace std;

void byValue(long long fee, int p) { fee -= fee * p / 100; cout << "  inside byValue: " << fee << '\n'; }
void byRef(long long& fee, int p)   { fee -= fee * p / 100; }
void byPtr(long long* fee, int p)   { *fee -= *fee * p / 100; }
long long byReturn(long long fee, int p) { return fee - fee * p / 100; }   // alternative

int main()
{
    long long a = 45000, b = 45000, c = 45000;
    byValue(a, 10);  cout << "by value   : " << a << '\n';
    byRef(b, 10);    cout << "by ref     : " << b << '\n';
    byPtr(&c, 10);   cout << "by pointer : " << c << '\n';

    // several concessions: merit 10%, sports 15%, single parent 25%
    long long fee = byReturn(byReturn(byReturn(45000, 10), 15), 25);
    cout << "all three concessions (return style): " << fee << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q1_three_ways.cpp -o q1_three_ways && ./q1_three_ways
  inside byValue: 40500
by value   : 45000
by ref     : 40500
by pointer : 40500
all three concessions (return style): 25819
```

Output screenshot –

q1_complete.cpp — code

```cpp
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;          // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;          // TODO 1 done: call by reference changes the caller's fee
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q1_complete.cpp -o q1_complete && ./q1_complete
after call by value : 45000
after call by reference: 40500
```

4. Inquiry – answers

- **Where each version stores its parameter:** by value – a new local variable in the callee's stack frame holding a copy (8 bytes for long long); by reference – no new object, the name fee is an alias of the caller's variable; by pointer – a new local pointer variable (8 bytes) holding the caller's address, and *fee reaches the caller's variable.
- **Why only two change the caller's fee:** only a reference or a pointer gives the function access to the caller's own variable. Call by value gives only a copy, destroyed on return. That is

exactly the hostel bug: the fee is correct inside the function (40500 is printed) but the stored fee stays 45000.

- **When to use a const reference:** when the function only reads a large object (Student, string, vector). void printReceipt(const Student& s) copies nothing and the compiler rejects any accidental change. For small types such as int or long long, pass by value.
- **Reference or pointer:** a reference cannot be null and needs no * or & at the call; a pointer can be null and &fee makes the change visible at the call site. Prefer a reference unless "no object" is a valid argument.

5. Design decision – change in place or return the new fee?

I choose to **return the new fee** (fee = applyConcession(fee, p);). The function has no side effect, so it can be tested and used for a "what-if" preview without touching the stored fee, and several concessions chain naturally: byReturn(byReturn(byReturn(45000,10),15),25) = 25819 (shown above). For a long long the returned copy costs 8 bytes. In-place update is better only for a large Student object that is updated often – then take Student&. Note that integer arithmetic truncates at each step (34425 × 25 / 100 = 8606.25 → 8606), so the order of the concessions must be fixed by the office rules.

6. Alternative concept and complexity analysis

Return-the-new-fee version

```
long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; }

fee = applyConcession(fee, 10);          // caller stores the result
```

S = sizeof(Student), for example about 1 KB.

Passing style	Time	Data copied: long long	Data copied: large Student	Remark
By value (or return a copy)	O(1) arithmetic; O(S) to copy an object	8 bytes	S bytes in, S bytes out	Safe and isolated; costly for big objects
By reference	O(1)	none (address only)	none (address only)	Changes the caller's object
By const reference	O(1)	none (address only)	none (address only)	Read only; best for large inputs
By pointer	O(1)	none (8-byte address)	none (8-byte address)	Same as reference, plus a null check

Q2 – Return by Reference and Its Dangers

1. Prediction (made before running)

Statement	Predicted	Reason
highest(marks,5) += 2	56 78 93 64 85	The returned reference is the array element itself.
highest(marks,5) = 95	56 78 95 64 85	A function returning a reference can be on the left of =.
int c = highestCopy(...); c += 2	array unchanged	The caller receives a copy; only the copy changes.

Statement	Predicted	Reason
int& r = bad();	compiler warning; using r is undefined	x is destroyed when bad() returns.
vector push_back when full	buffer moves, old address invalid	The vector allocates a larger block and frees the old one.

2. Code

q2_marks.cpp

```
#include <iostream>
#include <vector>
using namespace std;

int maxIdx(const int a[], int n)
{
   int k = 0;
   for (int i = 1; i < n; i++) if (a[i] > a[k]) k = i;
   return k;
}

int&       highest(int a[], int n)            { return a[maxIdx(a, n)]; }   // reference to the element
const int& highestConst(const int a[], int n) { return a[maxIdx(a, n)]; }   // read-only reference
int        highestCopy(const int a[], int n)  { return a[maxIdx(a, n)]; }   // copy of the value

void print(const char* t, const int a[], int n)
{
   cout << t; for (int i = 0; i < n; i++) cout << ' ' << a[i]; cout << '\n';
}

int main()
{
   int marks[5] = {56, 78, 91, 64, 85};
   print("start                :", marks, 5);

   highest(marks, 5) += 2;                  // moderation applied directly to the array element
   print("highest(...) += 2     :", marks, 5);

   highest(marks, 5) = 95;                  // call on the LEFT of =
   print("highest(...) = 95     :", marks, 5);

   int copy = highestCopy(marks, 5); copy += 2;  // copy: array untouched
   print("returned copy += 2    :", marks, 5);

   const int& cr = highestConst(marks, 5);      // read-only view; cr = 1;  would not compile
   cout << "const reference reads  : " << cr << '\n';

   vector<int> v = {56, 78, 91, 64, 85};
   int* p = &v[2];                          // address of the element a reference would refer to
   cout << "capacity before push_back: " << v.capacity() << '\n';
   v.push_back(70);                         // vector is full -> new, bigger buffer
   cout << "capacity after  push_back: " << v.capacity() << '\n';
   cout << "old reference still points into the vector? " << boolalpha << (p == &v[2]) << '\n';
```

```
}
```

q2_dangling.cpp

```
#include <iostream>
int& bad() { int x = 50; return x; }       // returns a reference to a local variable
int main() { int& r = bad(); std::cout << r << '\n'; }   // undefined behaviour
```

3. Output

Output screenshot

q2_marks.cpp — code

```
#include <iostream>
#include <vector>
using namespace std;

int maxIdx(const int a[], int n)
{
    int k = 0;
    for (int i = 1; i < n; i++) if (a[i] > a[k]) k = i;
    return k;
}

int&       highest(int a[], int n)            { return a[maxIdx(a, n)]; }   // reference to the element
const int& highestConst(const int a[], int n) { return a[maxIdx(a, n)]; }   // read-only reference
int        highestCopy(const int a[], int n)  { return a[maxIdx(a, n)]; }   // copy of the value

void print(const char* t, const int a[], int n)
{
    cout << t; for (int i = 0; i < n; i++) cout << ' ' << a[i]; cout << '\n';
}

int main()
{
    int marks[5] = {56, 78, 91, 64, 85};
    print("start                 :", marks, 5);

    highest(marks, 5) += 2;                       // moderation applied directly to the array element
    print("highest(...) += 2     :", marks, 5);

    highest(marks, 5) = 95;                       // call on the LEFT of =
    print("highest(...) = 95     :", marks, 5);

    int copy = highestCopy(marks, 5); copy += 2;  // copy: array untouched
    print("returned copy += 2    :", marks, 5);

    const int& cr = highestConst(marks, 5);       // read-only view; cr = 1;  would not compile
    cout << "const reference reads  : " << cr << '\n';

    vector<int> v = {56, 78, 91, 64, 85};
    int* p = &v[2];                               // address of the element a reference would refer to
    cout << "capacity before push_back: " << v.capacity() << '\n';
    v.push_back(70);                              // vector is full -> new, bigger buffer
    cout << "capacity after  push_back: " << v.capacity() << '\n';
    cout << "old reference still points into the vector? " << boolalpha << (p == &v[2]) << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q2_marks.cpp -o q2_marks && ./q2_marks
start                 : 56 78 91 64 85
highest(...) += 2     : 56 78 93 64 85
highest(...) = 95     : 56 78 95 64 85
returned copy += 2    : 56 78 95 64 85
const reference reads  : 95
capacity before push_back: 5
capacity after  push_back: 10
old reference still points into the vector? false
```

Output screenshot – Compiler message for q2_dangling.cpp (-Wall -Wextra) – the program was NOT run

q2_dangling.cpp – code

```
1  #include <iostream>
2  int& bad() { int x = 50; return x; }       // returns a reference to a local variable
3  int main() { int& r = bad(); std::cout << r << '\n'; }   // undefined behaviour
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q2_dangling.cpp -o q2_dangling
Q2/q2_dangling.cpp:2:33: warning: reference to local variable 'x' returned [-Wreturn-local-addr]
```

4. Explanation

- **Return by reference:** highest() returns an alias of the largest element, so highest(marks,5) += 2 moderates the mark in the array, and highest(marks,5) = 95 works because the call is an lvalue.

Return type	Caller gets	Can change the array element	Call on left of =	Risk
int&	alias of the element	yes	yes	dangling if the object dies
const int&	read-only alias	no (cr = 1 does not compile)	no	dangling if the object dies; no copy cost
int	independent copy	no (only the copy)	no	none; copy cost for big types

- **Dangling reference:** the compiler warns “reference to local variable ‘x’ returned”. x lives in bad()’s stack frame, which ends at return; r then refers to dead memory. Reading it may give 50, garbage, or crash – this is undefined behaviour, so the compiler may do anything.
- **Changed condition (std::vector that grows):** when push_back finds the vector full, it allocates a bigger block, moves the elements and frees the old block. In the output the capacity changed from 5 to 10 and the old address no longer matched (false). Every old reference, pointer and iterator is now dangling. Fix: keep an index, call reserve() first, or take the reference again after the vector has grown.

5. Alternative concept and complexity analysis (index or pointer instead of reference)

q2_alternatives.cpp

```
#include <iostream>
using namespace std;

int highestIndex(const int a[], int n)
{
    int k = 0;
    for (int i = 1; i < n; i++) if (a[i] > a[k]) k = i;
    return k;
}
int* highestPtr(int a[], int n) { return &a[highestIndex(a, n)]; }

int main()
{
    int marks[5] = {56, 78, 91, 64, 85};
    marks[highestIndex(marks, 5)] += 2;      // index style
```

```
    *highestPtr(marks, 5) += 2;            // pointer style (needs *)
    for (int m : marks) cout << m << ' ';
    cout << '\n';
}
```

Output screenshot –

q2_alternatives.cpp – code

```
1  #include <iostream>
2  using namespace std;
3
4  int highestIndex(const int a[], int n)
5  {
6      int k = 0;
7      for (int i = 1; i < n; i++) if (a[i] > a[k]) k = i;
8      return k;
9  }
10 int* highestPtr(int a[], int n) { return &a[highestIndex(a, n)]; }
11
12 int main()
13 {
14     int marks[5] = {56, 78, 91, 64, 85};
15     marks[highestIndex(marks, 5)] += 2;      // index style
16     *highestPtr(marks, 5) += 2;              // pointer style (needs *)
17     for (int m : marks) cout << m << ' ';
18     cout << '\n';
19 }
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q2_alternatives.cpp -o q2_alternatives && ./q2_alternatives
56 78 95 64 85
```

Return	Calling code	Time	Space	Safe if the array becomes a growing vector?
int&	highest(a,n) += 2;	O(n)	O(1)	No – invalid after reallocation
int (index)	a[highestIndex(a,n)] += 2;	O(n)	O(1)	Yes – an index stays valid after the buffer moves (until elements are erased or inserted before it)
int*	*highestPtr(a,n) += 2;	O(n)	O(1)	No – same problem as a reference

Q3 – Default Arguments and Function Overloading

1. Compiler error in the original design

q3_ambiguous.cpp

```
// the ORIGINAL design - the calls below do not compile
double charge(double weight);
double charge(double weight, bool express);
double charge(double weight, bool express = false, bool insured = false);

int main()
{
    charge(5.0);          // charge(double) or charge(double,bool=false,bool=false)?
```

```
    charge(5.0, true);    // charge(double,bool) or charge(double,bool,bool=false)?
}
```

Output screenshot – Compiler output (shortened)

q3_ambiguous.cpp – code

```
1  // the ORIGINAL design - the calls below do not compile
2  double charge(double weight);
3  double charge(double weight, bool express);
4  double charge(double weight, bool express = false, bool insured = false);
5
6  int main()
7  {
8      charge(5.0);          // charge(double) or charge(double,bool=false,bool=false)?
9      charge(5.0, true);    // charge(double,bool) or charge(double,bool,bool=false)?
10 }
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q3_ambiguous.cpp -o q3_ambiguous
Q3/q3_ambiguous.cpp:8:11: error: call of overloaded 'charge(double)' is ambiguous
Q3/q3_ambiguous.cpp:2:8: note: candidate: 'double charge(double)'
Q3/q3_ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool, bool)'
Q3/q3_ambiguous.cpp:9:11: error: call of overloaded 'charge(double, bool)' is ambiguous
Q3/q3_ambiguous.cpp:3:8: note: candidate: 'double charge(double, bool)'
Q3/q3_ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool, bool)'
```

Why: charge(5.0) matches both charge(double) and charge(double, bool=false, bool=false) equally well (exact match, defaults fill the rest). charge(5.0, true) matches both charge(double,bool) and the three-parameter version. With no better candidate the compiler reports an ambiguous call.

2. Prediction (made before running)

Call	Function selected	Reason
charge(2)	charge(int grams, …)	2 is an int: exact match.
charge(2.0f)	charge(double kg, …)	float → double is a promotion; float → int is a conversion, so promotion wins.
charge(2.5)	charge(double kg, …)	exact match.
charge(500, true)	charge(int grams, …)	int exact; true fills express.
charge(1.5, true, true)	charge(double kg, …)	all three arguments given.
charge(Parcel{3.0,true,false})	charge(const Parcel&)	only overload that accepts a Parcel.

3. Code (redesign)

q3_courier.cpp

```
#include <iostream>
using namespace std;

struct Parcel { double kg; bool express = false; bool insured = false; };

static double price(double kg, bool e, bool i) { return kg * 40 + (e ? 25 : 0) + (i ? 10 : 0); }

double charge(int grams, bool express = false, bool insured = false)
```

```
{ cout << "[int grams] ";  return price(grams / 1000.0, express, insured); }

double charge(double kg, bool express = false, bool insured = false)
{ cout << "[double kg] ";  return price(kg, express, insured); }

double charge(const Parcel& p)
{ cout << "[Parcel] ";     return price(p.kg, p.express, p.insured); }

#define SHOW(call) { cout << #call << " -> "; double r = call; cout << r << '\n'; }

int main()
{
   SHOW(charge(2))                    // int  : exact match
   SHOW(charge(2.0f))                  // float -> double is a promotion (beats float -> int)
   SHOW(charge(2.5))                   // double : exact match
   SHOW(charge(500, true))               // int grams + express
   SHOW(charge(1.5, true, true))          // kg + express + insured
   SHOW(charge(Parcel{3.0, true, false}))   // Parcel object
}
```

4. Output

Output screenshot – Output (matches the prediction; 2 g costs 0.08, 2 kg costs 80 at ₹40/kg)

q3_courier.cpp – code

```cpp
#include <iostream>
using namespace std;

struct Parcel { double kg; bool express = false; bool insured = false; };

static double price(double kg, bool e, bool i) { return kg * 40 + (e ? 25 : 0) + (i ? 10 : 0); }

double charge(int grams, bool express = false, bool insured = false)
{ cout << "[int grams] ";  return price(grams / 1000.0, express, insured); }

double charge(double kg, bool express = false, bool insured = false)
{ cout << "[double kg] ";  return price(kg, express, insured); }

double charge(const Parcel& p)
{ cout << "[Parcel] ";     return price(p.kg, p.express, p.insured); }

#define SHOW(call) { cout << #call << " -> "; double r = call; cout << r << '\n'; }

int main()
{
    SHOW(charge(2))                          // int  : exact match
    SHOW(charge(2.0f))                       // float -> double is a promotion (beats float -> int)
    SHOW(charge(2.5))                        // double : exact match
    SHOW(charge(500, true))                  // int grams + express
    SHOW(charge(1.5, true, true))            // kg + express + insured
    SHOW(charge(Parcel{3.0, true, false}))   // Parcel object
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q3_courier.cpp -o q3_courier && ./q3_courier
charge(2) -> [int grams] 0.08
charge(2.0f) -> [double kg] 80
charge(2.5) -> [double kg] 100
charge(500, true) -> [int grams] 45
charge(1.5, true, true) -> [double kg] 95
charge(Parcel{3.0, true, false}) -> [Parcel] 145
```

5. Inquiry – answers

- **How the compiler chooses:** for each argument it ranks the conversion needed: (1) exact match (including adding const or array-to-pointer), (2) promotion (float → double, char/short/bool → int), (3) standard conversion (int → double, long → int …), (4) user-defined conversion. The function that is at least as good on every argument and better on one wins; if two candidates tie, the call is ambiguous. Default arguments are filled in at compile time before this ranking.
- **Defaults from the right:** arguments are matched left to right, so a call can only leave out trailing arguments. f(int a = 1, int b) would give no way to omit a while supplying b, so the language requires defaults to start from the right.
- **Remaining hazard:** charge(2L) (a long) is still ambiguous because long → int and long → double are both conversions. The caller must write charge(2) or charge(2.0) – I confirmed this with the compiler.

6. Alternative concept and complexity analysis

Single Parcel structure, or a function template (sketch)

```
struct Parcel { double kg; bool express = false; bool insured = false; };
double charge(const Parcel& p);              // charge({2.5, true});

template <class W>                           // one template instead of two overloads
double charge(W w, bool e = false, bool i = false)
{ double kg = std::is_integral<W>::value ? w / 1000.0 : w;  return price(kg, e, i); }
```

Design	Overload resolution	Data copied per call	Adding a new option (e.g. fragile)
Overloads + default arguments	compile time, no run-time cost	8 bytes per double/int and 1 byte per bool	Add a parameter to every overload; more combinations, more risk of ambiguity
Parcel structure with optional fields	none – one function	by const reference: only an address (8 bytes)	Add one field with a default; no call site changes
Function template	compile time, one instantiation per type	same as overloads	Add one parameter in one place; type errors appear at instantiation

Q4 – Specifying a Class with an Array Inside It

1. Prediction (made before running)

Opening balance 5000; 12 transactions (deposit i × 100, but withdraw i × 50 when i is a multiple of 4). Only transactions 3 to 12 should remain on the mini-statement (the 11th and 12th push out the 1st and 2nd). Balance = 5000 + 5400 deposits − 1200 withdrawals = **9200**. The withdrawal of 100000 must be rejected.

2. Code

q4_account.cpp

```
#include <iostream>
using namespace std;
```

```
class SavingsAccount {
    int    accNo;
    double balance;
    double tx[10];              // last 10 transactions: + deposit, - withdrawal
    int    count;
    void record(double amt);       // defined outside
public:
    SavingsAccount(int a, double b) : accNo(a), balance(b), count(0) {}   // inside

    void deposit(double amt) { balance += amt; record(amt); }            // inside
    bool withdraw(double amt);                                           // outside
    void miniStatement() const;                                          // outside
};

void SavingsAccount::record(double amt)
{
    if (count == 10) {                          // 11th transaction: drop the oldest
        for (int i = 1; i < 10; i++) tx[i - 1] = tx[i];   // shift left, O(k)
        count = 9;
    }
    tx[count++] = amt;
}

bool SavingsAccount::withdraw(double amt)
{
    if (amt > balance) { cout << "Insufficient balance for " << amt << '\n'; return false; }
    balance -= amt;
    record(-amt);
    return true;
}

void SavingsAccount::miniStatement() const
{
    cout << "A/c " << accNo << "  last " << count << " transactions\n";
    for (int i = 0; i < count; i++)
        cout << ' ' << i + 1 << ". " << (tx[i] >= 0 ? "Deposit  " : "Withdraw ")
             << (tx[i] >= 0 ? tx[i] : -tx[i]) << '\n';
    cout << "Balance: " << balance << '\n';
}

int main()
{
    SavingsAccount a(1001, 5000);
    for (int i = 1; i <= 12; i++)
        if (i % 4 == 0) a.withdraw(i * 50); else a.deposit(i * 100);
    a.withdraw(100000);
    a.miniStatement();
}
```

3. Output

Output screenshot – Output (matches the prediction)

q4_account.cpp – code

```cpp
#include <iostream>
using namespace std;

class SavingsAccount {
    int    accNo;
    double balance;
    double tx[10];                // last 10 transactions: + deposit, - withdrawal
    int    count;
    void record(double amt);       // defined outside
public:
    SavingsAccount(int a, double b) : accNo(a), balance(b), count(0) {}   // inside

    void deposit(double amt) { balance += amt; record(amt); }             // inside
    bool withdraw(double amt);                                            // outside
    void miniStatement() const;                                           // outside
};

void SavingsAccount::record(double amt)
{
    if (count == 10) {                                // 11th transaction: drop the oldest
        for (int i = 1; i < 10; i++) tx[i - 1] = tx[i];   // shift left, O(k)
        count = 9;
    }
    tx[count++] = amt;
}

bool SavingsAccount::withdraw(double amt)
{
    if (amt > balance) { cout << "Insufficient balance for " << amt << '\n'; return false; }
    balance -= amt;
    record(-amt);
    return true;
}

void SavingsAccount::miniStatement() const
{
    cout << "A/c " << accNo << "  last " << count << " transactions\n";
    for (int i = 0; i < count; i++)
        cout << ' ' << i + 1 << ". " << (tx[i] >= 0 ? "Deposit  " : "Withdraw ")
             << (tx[i] >= 0 ? tx[i] : -tx[i]) << '\n';
    cout << "Balance: " << balance << '\n';
}

int main()
{
    SavingsAccount a(1001, 5000);
    for (int i = 1; i <= 12; i++)
        if (i % 4 == 0) a.withdraw(i * 50); else a.deposit(i * 100);
    a.withdraw(100000);
    a.miniStatement();
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q4_account.cpp -o q4_account && ./q4_account
Insufficient balance for 100000
A/c 1001  last 10 transactions
 1. Deposit  300
 2. Withdraw 200
 3. Deposit  500
 4. Deposit  600
 5. Deposit  700
 6. Withdraw 400
 7. Deposit  900
 8. Deposit  1000
 9. Deposit  1100
 10. Withdraw 600
Balance: 9200
```

4. Inquiry – answers

- **Why the array must be private:** if it were public, any code could rewrite or erase the history, or change count so it no longer matches the stored data. Keeping it private means only deposit() and withdraw() can add a transaction, and the balance and the history always agree.
- **Member functions inside and outside:** the constructor and deposit() are short and are defined inside the class (implicitly inline); withdraw(), miniStatement() and the private helper record() are defined outside using SavingsAccount::.
- **11th transaction – shifting:** record() moves elements 1…9 one place left and writes the new one at the end: 9 moves, O(k) time (k = capacity − 1), O(1) extra space.
- **11th transaction – circular index:** a head index marks the oldest entry; the new transaction overwrites that slot and head moves on by one (wrapping around): O(1) time, O(1) extra space. Order is restored when printing by reading from head.

5. Design decision – last 50 transactions

Replace the literal 10 by one named constant (static const int MAXTX = 50;) used for the array size, the full test and the loop; then **only that line changes** in either design. I would choose the **circular buffer**: the shifting version would then move 49 values per transaction (measured cost rises with capacity – see Q7), while the circular version stays O(1) at any capacity.

6. Alternative concept and complexity analysis

Alternatives (full tested versions are in Q7)

```
void add(int x) {                          // circular buffer, CAP = 10 or 50
   if (n < CAP) { a[(head + n) % CAP] = x; n++; }
   else         { a[head] = x; head = (head + 1) % CAP; }   // overwrite the oldest
}
// std::deque: if (d.size() == CAP) d.pop_front();  d.push_back(x);
```

Design	Time to add a transaction	Space	Measured (200,000 adds, CAP = 50)
Fixed array, shift left	O(k) – k elements shifted	O(k), fixed array	1.73 ms
Circular buffer (start index)	O(1)	O(k) + one int (head)	0.21 ms
std::deque (push_back / pop_front)	O(1) amortised	O(k) + block bookkeeping, heap allocation	0.46 ms

Q5 – Static Members and Arrays of Objects

1. Design derived from the principle

- **Which members are static:** issued (the number of seats given out) and CAPACITY belong to the whole bus, not to one ticket, so they exist once however many tickets are created. seat and name belong to each ticket and stay non-static.
- **Where a static data member is defined:** it is only declared in the class; it must be defined once, outside the class, in one source file (int BusTicket::issued = 0;). Without that definition the linker reports “undefined reference”. (static const int CAPACITY = 3; is an integral constant, so it can be given its value inside the class.)

- **Static member function:** count() has no this pointer, so it cannot use seat or name (non-static members belong to an object); it can use static members and is called as BusTicket::count() without any object.
- **Array of objects:** the constructor runs once for each element, in order. BusTicket has only an explicit one-argument constructor (no default constructor), so every element must be initialised in the braces: BusTicket trip[4] = {BusTicket("Asha"), …};.
- **Tickets created in a loop or function:** the constructor increments the static counter, but destroying a ticket does not decrement it, so count() means "seats issued so far", not "tickets alive".

2. Prediction (Predict the Output, made before running)

The static counter next is defined as 100 outside the class, so the first constructor makes it 101. counter[3] runs the constructor 3 times (ids 101, 102, 103) and extra once more (104): **4 constructor calls**. Predicted output: **101 103 104 4** (issued() = 104 − 100).

3. Code (Complete the Code: TODO 1 and TODO 2 done)

q5_bus.cpp

```
#include <iostream>
#include <string>

class BusTicket {
    static int issued;                  // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)              // TODO 1 done
            seat = ++issued;                // next seat number; otherwise seat stays 0 (waiting list)
    }
    static int count()
    {
        return issued;                  // TODO 2 done
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
    }
};

int BusTicket::issued = 0;                  // definition of the static data member

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"), BusTicket("Chitra"),
BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

4. Output

Output screenshot – Predict-the-output program (matches the prediction)

q5_predict.cpp — code

```cpp
#include <iostream>

class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' '
              << extra.get() << ' ' << Token::issued() << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q5_predict.cpp -o q5_predict && ./q5_predict
101 103 104 4
```

Output screenshot – q5_bus.cpp (matches the expected output; 4 tickets for 3 seats, Dev is on the waiting list)

q5_bus.cpp – code

```cpp
#include <iostream>
#include <string>

class BusTicket {
    static int issued;                      // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)              // TODO 1 done
            seat = ++issued;                // next seat number; otherwise seat stays 0 (waiting list)
    }
    static int count()
    {
        return issued;                      // TODO 2 done
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
    }
};

int BusTicket::issued = 0;                  // definition of the static data member

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"), BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q5_bus.cpp -o q5_bus && ./q5_bus
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5. Tickets created inside a function called twice

Only main part changed; the class is exactly as above

```cpp
void bookTwo(const std::string& a, const std::string& b)
{
    BusTicket t1(a), t2(b);                 // destroyed when the function returns
    t1.print(); t2.print();
    std::cout << "  inside function, issued = " << BusTicket::count() << '\n';
}

int main()
{
    bookTwo("Esha", "Farid");
    std::cout << "after call 1, issued = " << BusTicket::count() << '\n';
    bookTwo("Gita", "Hari");
    std::cout << "after call 2, issued = " << BusTicket::count() << '\n';
}
```

Output screenshot – Output

q5_function.cpp – code

```cpp
#include <iostream>
#include <string>

class BusTicket {
    static int issued;                      // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)              // TODO 1 done
            seat = ++issued;                // next seat number; otherwise seat stays 0 (waiting list)
    }
    static int count()
    {
        return issued;                      // TODO 2 done
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
    }
};

int BusTicket::issued = 0;                  // definition of the static data member

void bookTwo(const std::string& a, const std::string& b)
{
    BusTicket t1(a), t2(b);                 // destroyed when the function returns
    t1.print(); t2.print();
    std::cout << "  inside function, issued = " << BusTicket::count() << '\n';
}

int main()
{
    bookTwo("Esha", "Farid");
    std::cout << "after call 1, issued = " << BusTicket::count() << '\n';
    bookTwo("Gita", "Hari");
    std::cout << "after call 2, issued = " << BusTicket::count() << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q5_function.cpp -o q5_function && ./q5_function
Esha: Seat 1
Farid: Seat 2
  inside function, issued = 2
after call 1, issued = 2
Gita: Seat 3
Hari: Waiting
  inside function, issued = 3
after call 2, issued = 3
```

The tickets of the first call are destroyed when the function returns, but issued stays at 2; the second call therefore gets seat 3 and then a waiting-list ticket. The static counter lives for the whole program, independent of any object.

6. Alternative concept and complexity analysis (TicketOffice object)

q5_office.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class TicketOffice {                    // one counter per bus, no static data
   int issued = 0;
```

```
    const int cap;
public:
    explicit TicketOffice(int c) : cap(c) {}
    int next()        { return issued < cap ? ++issued : 0; }   // 0 = waiting list
    int count() const { return issued; }
};

class BusTicket {
    int seat; string name;
public:
    BusTicket(const string& n, TicketOffice& o) : seat(o.next()), name(n) {}
    void print() const { cout << name << ": " << (seat ? "Seat " + to_string(seat) :
string("Waiting")) << '\n'; }
};

int main()
{
    TicketOffice busA(3), busB(2);          // two buses at the same time
    BusTicket a1("Asha", busA), a2("Bala", busA);
    BusTicket b1("Chitra", busB), b2("Dev", busB), b3("Esha", busB);
    a1.print(); a2.print(); b1.print(); b2.print(); b3.print();
    cout << "Bus A issued " << busA.count() << ", Bus B issued " << busB.count() << '\n';
}
```

Output screenshot – Output – two buses at once

q5_office.cpp – code

```
#include <iostream>
#include <string>
using namespace std;

class TicketOffice {                    // one counter per bus, no static data
    int issued = 0;
    const int cap;
public:
    explicit TicketOffice(int c) : cap(c) {}
    int next()        { return issued < cap ? ++issued : 0; }   // 0 = waiting list
    int count() const { return issued; }
};

class BusTicket {
    int seat; string name;
public:
    BusTicket(const string& n, TicketOffice& o) : seat(o.next()), name(n) {}
    void print() const { cout << name << ": " << (seat ? "Seat " + to_string(seat) : string("Waiting")) << '\n'; }
};

int main()
{
    TicketOffice busA(3), busB(2);          // two buses at the same time
    BusTicket a1("Asha", busA), a2("Bala", busA);
    BusTicket b1("Chitra", busB), b2("Dev", busB), b3("Esha", busB);
    a1.print(); a2.print(); b1.print(); b2.print(); b3.print();
    cout << "Bus A issued " << busA.count() << ", Bus B issued " << busB.count() << '\n';
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q5_office.cpp -o q5_office && ./q5_office
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 1
Dev: Seat 2
Esha: Waiting
Bus A issued 2, Bus B issued 2
```

Design	Time	Space	Encapsulation	Two buses at once?
Static data member	O(1)	one counter	Counter is private to the class, one per program	No – all tickets share one counter
Global variable	O(1)	one counter	Weakest: any code can change it; name clashes	No (needs one global per bus)
TicketOffice object passed to each ticket	O(1)	one counter per office	Good: counter is private to the office; dependency is explicit	Yes – shown above (bus A and bus B)

Q6 – Friend Functions and Inline Functions

1. Design answers

- **Why a friend function:** minutesBetween(a, b) and earlier(a, b) treat both times equally. A friend is an ordinary function written f(a, b) that is allowed to read the private h and m of both objects, so no getters are needed.
- **Friend vs member:** a member is called a.f(b), has a this pointer and makes one object special; a friend is called like a normal function, has no this, and is not a member (friendship is not inherited, not mutual and not transitive). Both can read private data.
- **inline:** it is only a request to the compiler to put the function body at the call site instead of making a call. The compiler may ignore it (and at -O0 it does not inline). It suits tiny functions called millions of times; it also lets a function be defined in a header without a multiple-definition error.
- **If the members were public:** any code could set h = 99 or m = -5; the class could no longer guarantee a valid time, and changing the representation (for example to one minutes-since-midnight int) would break every user of the class.

2. Prediction (made before running)

Test pairs: (9:45, 13:10) → **205 false**; the same time twice (8:05, 8:05) → **0 false**; either side of noon (11:50, 12:20) → **30 true**. Timing: at -O0 no version is inlined, so all three are about equal; at -O2 the inline version should be faster, but only by a small amount because each function body is tiny.

3. Code (Complete the Code: TODO 1 and TODO 2 done)

q6_time.cpp

```
#include <iostream>
#include <cstdlib>

class Time {
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend int minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
   return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m));   // TODO 1 done
```

```
}

bool earlier(const Time& a, const Time& b)
{
   return a.h * 60 + a.m < b.h * 60 + b.m;                // TODO 2 done
}

int main()
{
   Time departure(9, 45), arrival(13, 10);
   std::cout << minutesBetween(departure, arrival) << ' '
             << std::boolalpha << earlier(arrival, departure) << '\n';

   Time x(11, 50), y(12, 20), z(8, 5);                    // extra tests
   std::cout << minutesBetween(z, z) << ' ' << earlier(z, z) << '\n';   // same time twice
   std::cout << minutesBetween(x, y) << ' ' << earlier(x, y) << '\n';   // either side of noon
}
```

4. Output

Output screenshot – Output (line 1 matches the expected output "205 false"; lines 2–3 are my extra tests)

q6_time.cpp – code

```cpp
#include <iostream>
#include <cstdlib>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m));   // TODO 1 done
}

bool earlier(const Time& a, const Time& b)
{
    return a.h * 60 + a.m < b.h * 60 + b.m;                 // TODO 2 done
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';

    Time x(11, 50), y(12, 20), z(8, 5);                     // extra tests
    std::cout << minutesBetween(z, z) << ' ' << earlier(z, z) << '\n';   // same time twice
    std::cout << minutesBetween(x, y) << ' ' << earlier(x, y) << '\n';   // either side of noon
}
```

Terminal

```
$ g++ -std=c++17 -Wall -Wextra q6_time.cpp -o q6_time && ./q6_time
205 false
0 false
30 true
```

5. Growth investigation – 10,000,000 calls

q6_timing.cpp

```cpp
#include <chrono>
#include <iostream>
using namespace std;

class Time {
   int h, m;
public:
   Time(int hh = 0, int mm = 0) : h(hh), m(mm) {}
   friend inline int fInline(const Time& a, const Time& b) { return (a.h*60+a.m) - (b.h*60+b.m);
}
   friend __attribute__((noinline)) int fNoInline(const Time& a, const Time& b)
   { return (a.h*60+a.m) - (b.h*60+b.m); }
   int minutesTo(const Time& b) const { return (b.h*60+b.m) - (h*60+m); }   // member,
implicitly inline
};
```

```
const int N = 10000000;
Time t[1024];

#define TIME(name, call)                                                \
   { auto t0 = chrono::steady_clock::now(); long long s = 0;            \
     for (int i = 0; i < N; i++) { const Time &x = t[i & 1023], &y = t[(i + 7) & 1023]; s += call; } \
     cout << name << ": " << chrono::duration<double, milli>(chrono::steady_clock::now() -
t0).count() \
          << " ms (check " << s << ")\n"; }

int main()
{
   for (int i = 0; i < 1024; i++) t[i] = Time(i % 24, i % 60);
   TIME("friend, inline    ", fInline(x, y))
   TIME("friend, not inline", fNoInline(x, y))
   TIME("member function   ", x.minutesTo(y))
}
```

Version (10,000,000 calls)	**-O0 (ms)**	**-O2 (ms)**
Friend function, inline	27.8	10.6
Friend function, forced not inline (noinline attribute)	27.7	12.6
Member function (defined in class, implicitly inline)	27.2	9.7

Output screenshot – timing at -O0

q6_timing.cpp – code

```cpp
#include <chrono>
#include <iostream>
using namespace std;

class Time {
    int h, m;
public:
    Time(int hh = 0, int mm = 0) : h(hh), m(mm) {}
    friend inline int fInline(const Time& a, const Time& b) { return (a.h*60+a.m) - (b.h*60+b.m); }
    friend __attribute__((noinline)) int fNoInline(const Time& a, const Time& b)
    { return (a.h*60+a.m) - (b.h*60+b.m); }
    int minutesTo(const Time& b) const { return (b.h*60+b.m) - (h*60+m); }   // member, implicitly inline
};

const int N = 10000000;
Time t[1024];

#define TIME(name, call)                                                    \
    { auto t0 = chrono::steady_clock::now(); long long s = 0;               \
      for (int i = 0; i < N; i++) { const Time &x = t[i & 1023], &y = t[(i + 7) & 1023]; s += call; } \
      cout << name << ": " << chrono::duration<double, milli>(chrono::steady_clock::now() - t0).count() \
           << " ms (check " << s << ")\n"; }

int main()
{
    for (int i = 0; i < 1024; i++) t[i] = Time(i % 24, i % 60);
    TIME("friend, inline    ", fInline(x, y))
    TIME("friend, not inline", fNoInline(x, y))
    TIME("member function   ", x.minutesTo(y))
}
```

Terminal

```
$ g++ -std=c++17 -O0 q6_timing.cpp -o q6_O0 && ./q6_O0
friend, inline    : 47.5017 ms (check -7000)
friend, not inline: 27.2782 ms (check -7000)
member function   : 26.7368 ms (check 7000)
```

Output screenshot – timing at -O2

q6_timing.cpp – code

```cpp
#include <chrono>
#include <iostream>
using namespace std;

class Time {
    int h, m;
public:
    Time(int hh = 0, int mm = 0) : h(hh), m(mm) {}
    friend inline int fInline(const Time& a, const Time& b) { return (a.h*60+a.m) - (b.h*60+b.m); }
    friend __attribute__((noinline)) int fNoInline(const Time& a, const Time& b)
    { return (a.h*60+a.m) - (b.h*60+b.m); }
    int minutesTo(const Time& b) const { return (b.h*60+b.m) - (h*60+m); }   // member, implicitly inline
};

const int N = 10000000;
Time t[1024];

#define TIME(name, call)                                                    \
    { auto t0 = chrono::steady_clock::now(); long long s = 0;               \
      for (int i = 0; i < N; i++) { const Time &x = t[i & 1023], &y = t[(i + 7) & 1023]; s += call; } \
      cout << name << ": " << chrono::duration<double, milli>(chrono::steady_clock::now() - t0).count() \
           << " ms (check " << s << ")\n"; }

int main()
{
    for (int i = 0; i < 1024; i++) t[i] = Time(i % 24, i % 60);
    TIME("friend, inline    ", fInline(x, y))
    TIME("friend, not inline", fNoInline(x, y))
    TIME("member function   ", x.minutesTo(y))
}
```

Terminal

```
$ g++ -std=c++17 -O2 q6_timing.cpp -o q6_O2 && ./q6_O2
friend, inline    : 9.70542 ms (check -7000)
friend, not inline: 12.5317 ms (check -7000)
member function   : 9.6151 ms (check 7000)
```

Median of 3 runs on my machine (g++ 13.3). Your numbers will differ; repeat the runs on your own computer and record them.

- **-O0:** all three are the same (about 27 ms) – with no optimisation the compiler does not inline, so the inline keyword changes nothing.
- **-O2:** the inlined versions take about 10 ms against about 12.6 ms for the forced call: roughly 20% faster, i.e. about 0.2 ns saved per call. The gain is real but small, because the function body is tiny and the call is well predicted by the CPU. The member function is as fast as the inline friend – friend and member generate the same code.

6. Deployment decision

Keep the two **friend functions**. They are part of the Time interface, they cost nothing after inlining, and they avoid adding public getters for h and m that would expose the representation to everybody. If Time's representation changes, only Time and its friends need editing. Member functions would be equally good (identical speed); public getters would be the weakest choice for encapsulation.

7. Alternative concept and complexity analysis

Member-function / getter versions

```
class Time {
   int h, m;
public:
   int  minutesTo(const Time& b) const { return abs((b.h*60+b.m) - (h*60+m)); }   //
a.minutesTo(b)
```

```
    bool isEarlier(const Time& b)  const { return h*60+m < b.h*60+b.m; }          // a.isEarlier(b)
    int getH() const { return h; }  int getM() const { return m; }               // getter version
};
```

Design	Time per call	Measured -O0 / -O2	Encapsulation
Friend functions	O(1)	27.8 / 10.6 ms (inline)	Small controlled access to private data; symmetric f(a,b) syntax
Member functions	O(1)	27.2 / 9.7 ms	Natural: private data stays private; asymmetric a.f(b) syntax
Public getters + free functions	O(1)	same after inlining	Weakest: every user can read h and m

Q7 – Diagnose, Redesign and Validate

Program selected: Q4 – the SavingsAccount mini-statement that keeps its last transactions in a fixed array and shifts the array left when it is full. Q4 itself asks for the last 50 transactions, and a bank may ask for more, so the limitation matters as the requirement grows.

A. Diagnose the limitation

- **Where it slows down:** every transaction after the array is full moves all the other elements one place left. For 200,000 transactions the cost per transaction rose from **6.9 ns (CAP = 10) to 260 ns (CAP = 10,000)** – 38 times more for 1000 times more capacity (the rise is less than 1000× only because the compiler turns the loop into a fast block copy).
- **Cause:** the design keeps the oldest transaction at position 0, so adding one transaction physically moves k − 1 others: O(k) time per transaction.

B. Design objective

- Same contents and order as the original (checksum test must say “identical”).
- Time per transaction must not depend on capacity: time at CAP = 10,000 within 1.5× of the time at CAP = 10.
- At least 20× faster than the original at CAP = 1000, with no more than one extra int of memory.

C. Design of the revised solution

- **Change:** store the history as a **circular buffer** – add one data member head (index of the oldest transaction). A new transaction overwrites the oldest slot when full, and head moves on by one with wrap-around. The array itself never moves.
- **C++ feature used:** an array inside a class with index arithmetic in member functions (at(i) translates the i-th oldest to the physical slot (head + i) % CAP); a class template makes the capacity a compile-time constant.
- **Extra memory:** one int. **Trade-offs:** the data is no longer in time order in memory (reading must go through at(i)), wrap-around logic is more error-prone, and the code is longer than the simple shift.

Pseudocode

```
class Circular {                    // pseudocode of the redesign
    int a[CAP]; int head = 0, n = 0;
```

```
    add(x):  if n < CAP:  a[(head + n) % CAP] = x;  n++
             else:        a[head] = x;  head = (head + 1) % CAP
    at(i):   return a[(head + i) % CAP]     // i = 0 is the oldest
}
```

D. Prediction (made before implementing, from the complexity)

The original is O(CAP) per transaction and the redesign O(1), so the circular version should take about the same time at every capacity, and the speed-up should grow with capacity: at least **2× at CAP = 10, 20× at CAP = 1000 and 100× at CAP = 10,000**. Measure: milliseconds for 200,000 transactions (-O2). I expect std::deque to be O(1) too but slower than the circular buffer because of its heap blocks and indirection.

E. Implementation and validation

q7_redesign.cpp

```
#include <chrono>
#include <deque>
#include <iostream>
using namespace std;

// ORIGINAL: fixed array, shift left when full - O(CAP) per transaction
template <int CAP> struct Shift {
    int a[CAP]; int n = 0;
    void add(int x) {
        if (n == CAP) { for (int i = 1; i < CAP; i++) a[i - 1] = a[i]; n = CAP - 1; }
        a[n++] = x;
    }
    int  at(int i) const { return a[i]; }
    int  size() const { return n; }
};

// REDESIGN: circular buffer with a head index - O(1) per transaction, same memory
template <int CAP> struct Circ {
    int a[CAP]; int head = 0, n = 0;
    void add(int x) {
        if (n < CAP) { int p = head + n; if (p >= CAP) p -= CAP; a[p] = x; n++; }
        else         { a[head] = x; if (++head == CAP) head = 0; }
    }
    int  at(int i) const { int p = head + i; if (p >= CAP) p -= CAP; return a[p]; }
    int  size() const { return n; }
};

// ALTERNATIVE: std::deque
template <int CAP> struct Dq {
    deque<int> d;
    void add(int x) { if ((int)d.size() == CAP) d.pop_front(); d.push_back(x); }
    int  at(int i) const { return d[i]; }
    int  size() const { return (int)d.size(); }
};

template <class S> double bench(long long N, long long& check)
{
    static S s;                              // static: keeps big arrays off the stack
```

```
    s = S();
    auto t0 = chrono::steady_clock::now();
    for (long long i = 0; i < N; i++) s.add((int)i);
    double ms = chrono::duration<double, milli>(chrono::steady_clock::now() - t0).count();
    check = 0;                              // order-sensitive checksum of the stored transactions
    for (int i = 0; i < s.size(); i++) check = check * 31 + s.at(i);
    return ms;
}

template <int CAP> void row(long long N)
{
    long long c1, c2, c3;
    double t1 = bench<Shift<CAP>>(N, c1), t2 = bench<Circ<CAP>>(N, c2), t3 =
bench<Dq<CAP>>(N, c3);
    printf("%6d | %10.2f | %10.2f | %10.2f | %7.1fx | %s\n", CAP, t1, t2, t3, t1 / t2,
           (c1 == c2 && c2 == c3) ? "identical" : "DIFFERENT");
}

int main()
{
    const long long N = 200000;
    cout << "200,000 transactions; time in ms\n";
    cout << "   CAP |  shift (ms) | circular(ms)| deque (ms) | speed-up | contents\n";
    row<10>(N); row<50>(N); row<1000>(N); row<10000>(N);
}
```

Output screenshot – Output (same program, same 200,000 transactions for all three versions)

q7_redesign.cpp — code

```cpp
#include <chrono>
#include <deque>
#include <iostream>
using namespace std;

// ORIGINAL: fixed array, shift left when full - O(CAP) per transaction
template <int CAP> struct Shift {
    int a[CAP]; int n = 0;
    void add(int x) {
        if (n == CAP) { for (int i = 1; i < CAP; i++) a[i - 1] = a[i]; n = CAP - 1; }
        a[n++] = x;
    }
    int  at(int i) const { return a[i]; }
    int  size() const { return n; }
};

// REDESIGN: circular buffer with a head index - O(1) per transaction, same memory
template <int CAP> struct Circ {
    int a[CAP]; int head = 0, n = 0;
    void add(int x) {
        if (n < CAP) { int p = head + n; if (p >= CAP) p -= CAP; a[p] = x; n++; }
        else         { a[head] = x; if (++head == CAP) head = 0; }
    }
    int  at(int i) const { int p = head + i; if (p >= CAP) p -= CAP; return a[p]; }
    int  size() const { return n; }
};

// ALTERNATIVE: std::deque
template <int CAP> struct Dq {
    deque<int> d;
    void add(int x) { if ((int)d.size() == CAP) d.pop_front(); d.push_back(x); }
    int  at(int i) const { return d[i]; }
    int  size() const { return (int)d.size(); }
};

template <class S> double bench(long long N, long long& check)
{
    static S s;                                  // static: keeps big arrays off the stack
    s = S();
    auto t0 = chrono::steady_clock::now();
    for (long long i = 0; i < N; i++) s.add((int)i);
    double ms = chrono::duration<double, milli>(chrono::steady_clock::now() - t0).count();
    check = 0;                                   // order-sensitive checksum of the stored transactions
    for (int i = 0; i < s.size(); i++) check = check * 31 + s.at(i);
    return ms;
}

template <int CAP> void row(long long N)
{
    long long c1, c2, c3;
    double t1 = bench<Shift<CAP>>(N, c1), t2 = bench<Circ<CAP>>(N, c2), t3 = bench<Dq<CAP>>(N, c3);
    printf("%6d | %10.2f | %10.2f | %10.2f | %7.1fx | %s\n", CAP, t1, t2, t3, t1 / t2,
           (c1 == c2 && c2 == c3) ? "identical" : "DIFFERENT");
}

int main()
{
    const long long N = 200000;
    cout << "200,000 transactions; time in ms\n";
    cout << "   CAP |  shift (ms) | circular(ms)| deque (ms) | speed-up | contents\n";
    row<10>(N); row<50>(N); row<1000>(N); row<10000>(N);
}
```

Terminal

```
$ g++ -std=c++17 -O2 -Wall -Wextra q7_redesign.cpp -o q7 && ./q7
200,000 transactions; time in ms
   CAP |  shift (ms) | circular(ms)| deque (ms) | speed-up | contents
    10 |       1.38 |       0.22 |       0.43 |     6.1x | identical
    50 |       1.73 |       0.21 |       0.46 |     8.3x | identical
  1000 |       6.93 |       0.24 |       0.50 |    28.7x | identical
 10000 |      52.11 |       0.27 |       0.74 |   196.5x | identical
```

CAP	Original shift: ns per transaction	Circular: ns per transaction	Speed-up	Predicted
10	6.9	1.1	6.1×	≥ 2×

CAP	Original shift: ns per transaction	Circular: ns per transaction	Speed-up	Predicted
50	8.7	1.1	8.3×	–
1000	34.7	1.2	28.7×	≥ 20×
10,000	260.6	1.4	196.5×	≥ 100×

F. Defend the final design

- **Objective met:** contents are identical in all cases; circular time is flat (0.22 ms at CAP = 10, 0.27 ms at CAP = 10,000, within 1.5×); speed-up was 28.7× at CAP = 1000 and 196× at CAP = 10,000; extra memory is one int. The predictions held.
- **std::deque:** also flat (0.43 – 0.74 ms) and the simplest to write, but 2–3× slower than the circular buffer and it allocates on the heap. A good choice when the capacity is unknown or can grow.
- **New costs:** index translation in every read, wrap-around logic to test, a less obvious class.
- **Where the original is still better:** for CAP = 10 the shifting version costs about 7 ns per transaction – meaningless at ATM speed – and its array stays in order, so printing and debugging are trivial. The redesign is worth the extra complexity only when the capacity is large or the transaction rate is high.